

Blocco 12 - Capstone query design

Progettare una query completa, spiegabile e verificabile

Relational Databases & SQL

30 min teoria + 90 min pratica

1. Rappresentazione iniziale

Partiamo da una soluzione semplice e concreta, prima di introdurre il modello generale.

Perché questo blocco

Problema didattico

La richiesta finale è volutamente simile a un lavoro reale: una domanda business incompleta, più fonti dati, metriche, filtri, ranking e controlli. La sfida è trasformarla in una soluzione difendibile.

Idea guida

Prima osserviamo una rappresentazione naturale, poi ne discutiamo i limiti e solo dopo formalizziamo.

Richiesta business ambigua

Situazione concreta

Una richiesta finale arriva spesso come frase: “mostrami i segmenti migliori”. Prima del codice bisogna trasformarla in specifica verificabile.

Richiesta iniziale

Voglio capire quali segmenti stanno performando meglio negli ultimi mesi.

Limiti della rappresentazione iniziale

- ▶ non definisce metrica;
- ▶ non definisce periodo;
- ▶ non definisce stati inclusi;
- ▶ non definisce granularità;
- ▶ non dice quali controlli servono.

2. Soluzione più generale

Riorganizziamo l'informazione in una forma più flessibile e controllabile.

Da richiesta a query design

Principio

La soluzione professionale passa da specifica, assunzioni, CTE, implementazione, controlli e spiegazione finale.

Esempio di riorganizzazione

- ▶ metrica: ricavo valido;
- ▶ periodo: mese ordine;
- ▶ granularità: mese e segmento;
- ▶ output: ranking e quota percentuale;
- ▶ controlli: totali e cardinalità.

Processo di trasformazione

- ➊ riscrivere la domanda;
- ➋ dichiarare input e output;
- ➌ scegliere metrica e popolazione;
- ➍ progettare passaggi intermedi;
- ➎ aggiungere controlli e limiti.

Analogia o caso esplicativo

Un ingegnere non consegna solo il numero finale: descrive ipotesi, metodo, calcoli, verifiche e limiti. Il capstone richiede lo stesso: SQL più ragionamento.

3. Formalizzazione

Diamo un nome preciso ai concetti emersi dall'esempio.

Formalizzazione: specifica e verifica

- ▶ specifica tecnica;
- ▶ assunzioni;
- ▶ granularità;
- ▶ query finale;
- ▶ query di controllo;
- ▶ spiegazione dei limiti.

- ▶ **Specifica:** traduzione precisa della domanda business.
- ▶ **Assunzione:** decisione dichiarata dove il testo è ambiguo.
- ▶ **Query design:** sequenza di passaggi logici prima del codice.
- ▶ **Controllo:** query secondaria che verifica un aspetto del risultato.
- ▶ **Spiegabilit :** capacit  di difendere ogni clausola.

Come riconoscere una buona soluzione

- ▶ la specifica è chiara prima del codice;
- ▶ ogni CTE ha scopo e granularità leggibili;
- ▶ il risultato finale risponde alla domanda dichiarata;
- ▶ i controlli confermano popolazione, totali e cardinalità.

4. Esempio guidato

Applichiamo il modello generale a un caso piccolo, leggendo ogni passaggio.

Esempio: situazione iniziale

Domanda o frase di dominio

Domanda: “Per ogni mese, trovare i migliori paesi per ricavo valido, con ranking e quota percentuale sul totale mese.”

Esempio: ragionamento

- ▶ validità: escludere ordini cancellati e rimborsati;
- ▶ granularità base: ordine o riga ordine, poi mese-paese;
- ▶ metrica: ricavo lordo o netto, dichiarato;
- ▶ window function: ranking e quota sul totale mese.

Esempio completo: struttura capstone

```
WITH country_month AS (  
    SELECT date_trunc('month', r.order_date)::date AS month,  
           c.country,  
           round(sum(r.gross_revenue), 2) AS revenue  
    FROM order_revenue r  
    JOIN customers c ON c.customer_id = r.customer_id  
    WHERE r.status NOT IN ('cancelled', 'refunded')  
    GROUP BY month, c.country  
) , ranked AS (  
    SELECT month, country, revenue,  
           rank() OVER (PARTITION BY month ORDER BY revenue DESC, country) AS country_rank,  
           round(100 * revenue / sum(revenue) OVER (PARTITION BY month), 2) AS pct_month_revenue  
    FROM country_month  
)  
SELECT month, country, revenue, country_rank, pct_month_revenue  
FROM ranked  
WHERE country_rank <= 3  
ORDER BY month, country_rank, country;
```

Errori frequenti da prevenire

- ▶ iniziare a scrivere SQL senza specifica;
- ▶ non dichiarare stati inclusi e data di riferimento;
- ▶ costruire una query monolitica non verificabile;
- ▶ non controllare denominatori e righe perse nei join;
- ▶ presentare solo il risultato senza assunzioni.

5. Pratica guidata

Ripetiamo lo stesso processo su un problema diverso: rappresentazione iniziale, limiti, riorganizzazione, soluzione.

Esercitazione: problema informale

Contesto

La direzione chiede un report finale per capire quali paesi e segmenti contribuiscono maggiormente al ricavo, con trend, ranking e controlli di affidabilità.

Esercitazione: specifica corretta

- ▶ input: schema training, vista `order_revenue`, clienti e prodotti;
- ▶ output: query finale, controlli, breve spiegazione delle assunzioni;
- ▶ vincolo: usare CTE per separare passaggi logici;
- ▶ vincolo: includere almeno una window function e due query di controllo.

Esercitazione: definizione della soluzione

- ① scrivere prima la specifica in cinque righe;
- ② costruire CTE base per ordini validi e ricavi;
- ③ aggregare alla granularità richiesta;
- ④ applicare ranking o percentuali con window function;
- ⑤ preparare controlli su totali e numero righe.

Implementazione completa: report finale (1/2)

```
SET search_path TO training;

WITH valid_revenue AS (
    SELECT r.order_id, r.customer_id, r.order_date, r.channel, r.gross_revenue
    FROM order_revenue r
    WHERE r.status NOT IN ('cancelled', 'refunded')
), segment_month AS (
    SELECT date_trunc('month', v.order_date)::date AS month,
           c.segment,
           count(*) AS orders,
           round(sum(v.gross_revenue), 2) AS revenue
    FROM valid_revenue v
    JOIN customers c ON c.customer_id = v.customer_id
    GROUP BY month, c.segment
), ranked AS (
    SELECT month, segment, orders, revenue,
           rank() OVER (PARTITION BY month ORDER BY revenue DESC, segment) AS segment_rank,
           round(100 * revenue / sum(revenue) OVER (PARTITION BY month), 2) AS pct_month_revenue
    FROM segment_month
)
SELECT month, segment, orders, revenue, segment_rank, pct_month_revenue
FROM ranked
WHERE segment_rank <= 3
ORDER BY month, segment_rank, segment;
```


Implementazione completa: report finale (2/2)

```
WITH valid_revenue AS (  
    SELECT * FROM order_revenue  
    WHERE status NOT IN ('cancelled', 'refunded')  
)  
SELECT count(*) AS valid_orders,  
       round(sum(gross_revenue), 2) AS valid_revenue  
FROM valid_revenue;  
  
SELECT count(*) AS customers,  
       count(DISTINCT customer_id) AS distinct_customers  
FROM customers;
```

Esercitazione: organizzazione dei 90 minuti

- ▶ 20 min specifica e assunzioni;
- ▶ 25 min CTE base e aggregazioni;
- ▶ 25 min ranking, percentuali e query finale;
- ▶ 20 min controlli;
- ▶ 10 min presentazione e discussione.

- ▶ confrontare almeno due soluzioni quando esistono alternative;
- ▶ chiedere quali assunzioni cambierebbero la soluzione;
- ▶ separare errori di sintassi, errori di modello ed errori di interpretazione.

- ▶ il capstone valuta metodo, non solo sintassi;
- ▶ una query complessa si costruisce per passaggi verificabili;
- ▶ una soluzione professionale dichiara assunzioni, controlli e limiti.